

autoencoder_hybrids

October 6, 2025

1 Autoencoder for Fashion-MNIST → CIFAR-100

1.1 Class Centroids, Average Images & Hybrids

MSc HPC-AI mini-project (DL) — MINES Paris PSL

1.2 1. Project Overview & Objectives

Goals: - Train Convolutional Autoencoder (CAE) on Fashion-MNIST (grayscale, 10 classes) - Compute class centroids in latent space and class average images in pixel space - Generate hybrid images via centroid interpolation (e.g., T-shirt Sneaker) - Extend pipeline to CIFAR-100 subset (~10 classes): bus, rocket, bicycle, motorcycle, etc.

Note: CIFAR-100 does not include airplane/car classes. We use bus rocket as example hybrid pair.

1.3 2. Setup, Paths, and Reproducibility

```
[ ]: import sys
      sys.path.insert(0, '..')

      import torch
      import numpy as np
      import matplotlib.pyplot as plt
      from pathlib import Path

      from src.utils import set_seeds, get_device, ensure_dirs
      from src.data import fashion_loaders, cifar100_loaders, CIFAR100_SUBSET, DATA_DIR
      from src.model import ConvAE
      from src.train import fit
      from src.latent import encode_dataset, class_centroids, class_average_images
      from src.hybrids import interpolate, decode_latents, offset_hybrids
      from src.viz import grid, save_image, tsne_or_pca

      print(f"PyTorch version: {torch.__version__}")
      print(f"CUDA available: {torch.cuda.is_available()}")
      print(f"MPS available: {torch.backends.mps.is_available()}")
```

```

set_seeds(42)
device = get_device()
print(f"Using device: {device}")

ensure_dirs('figures', 'checkpoints', 'artifacts')

# Args
fashion_args = {
    'batch_size': 128,
    'lr': 1e-3,
    'epochs': 15,
    'latent_dim': 64,
    'num_workers': 0,
    'img_size': 28
}

cifar_args = {
    'batch_size': 128,
    'lr': 1e-3,
    'epochs': 20,
    'latent_dim': 128,
    'num_workers': 0,
    'img_size': 32
}

```

1.4 3. Data: Fashion-MNIST — Preparation & Preview

Using local data at `./data/fashionmnist/raw` if exists, otherwise download.

```

[ ]: # Check if local Fashion-MNIST exists
fmnist_path = DATA_DIR / 'fashionmnist' / 'raw'
if fmnist_path.exists():
    print(f" Found local Fashion-MNIST at {fmnist_path}")
else:
    print("Downloading Fashion-MNIST...")

train_loader, val_loader = fashion_loaders(
    batch_size=fashion_args['batch_size'],
    num_workers=fashion_args['num_workers']
)

print(f"Train samples: {len(train_loader.dataset)}")
print(f"Val samples: {len(val_loader.dataset)}")

# Fashion-MNIST class names
fashion_classes = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
                  'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

```

```
[ ]: # Preview samples
sample_batch = next(iter(train_loader))[0][:100]
sample_grid = grid(sample_batch, nrow=10)
save_image(sample_grid, 'figures/fashion_samples.png')

plt.figure(figsize=(10, 10))
plt.imshow(sample_grid, cmap='gray')
plt.axis('off')
plt.title('Fashion-MNIST Samples')
plt.tight_layout()
plt.savefig('figures/fashion_samples_preview.png', dpi=100, bbox_inches='tight')
plt.show()
```

1.5 4. Model: Convolutional Autoencoder (Fashion-MNIST)

Architecture: - Encoder: Conv(1→32) → Conv(32→64) → Linear(64*7*7 → latent_dim) - Decoder: Linear → ConvTranspose(64→32) → ConvTranspose(32→1) → Tanh - Weight init: Kaiming normal

```
[ ]: model_fashion = ConvAE(
    in_ch=1,
    latent_dim=fashion_args['latent_dim'],
    img_size=fashion_args['img_size']
).to(device)

print(model_fashion)
print(f"\nTotal params: {sum(p.numel() for p in model_fashion.parameters()):,}")

# Sanity check
with torch.no_grad():
    x_test = torch.randn(2, 1, 28, 28).to(device)
    z_test = model_fashion.encode(x_test)
    recon_test = model_fashion.decode(z_test)
    print(f"Input: {x_test.shape} → Latent: {z_test.shape} → Recon: {recon_test.
    ↪shape}")
```

1.6 5. Training the Autoencoder (Fashion-MNIST)

Loss: MSE

Optimizer: Adam (lr=1e-3, weight_decay=1e-5)

Epochs: 15

Batch size: 128

Checkpoint: Best val loss

```
[ ]: history_fashion = fit(
    model_fashion,
    train_loader,
    val_loader,
```

```

device,
epochs=fashion_args['epochs'],
lr=fashion_args['lr'],
ckpt_path='checkpoints/best_fashion_cae.pt'
)

# Plot loss curves
plt.figure(figsize=(8, 4))
plt.plot(history_fashion['train_loss'], label='Train')
plt.plot(history_fashion['val_loss'], label='Val')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.legend()
plt.title('Fashion-MNIST Training')
plt.grid(True, alpha=0.3)
plt.savefig('figures/fashion_loss.png', dpi=100, bbox_inches='tight')
plt.show()

```

```

[ ]: # Load best model
model_fashion.load_state_dict(torch.load('checkpoints/best_fashion_cae.pt'))
model_fashion.eval()
print(" Loaded best checkpoint")

```

1.7 6. Reconstructions & Class Average Images

```

[ ]: # Reconstruction grid
val_batch = next(iter(val_loader))[0][:50].to(device)
with torch.no_grad():
    recon_batch = model_fashion(val_batch)

combined = torch.cat([val_batch, recon_batch], dim=0)
recon_grid_img = grid(combined, nrow=10)
save_image(recon_grid_img, 'figures/fashion_recon_grid.png')

plt.figure(figsize=(10, 8))
plt.imshow(recon_grid_img, cmap='gray')
plt.title('Top: Original | Bottom: Reconstructions')
plt.axis('off')
plt.tight_layout()
plt.show()

```

```

[ ]: # Class average images in pixel space
# Collect indices per class from val set
val_dataset = val_loader.dataset.dataset.dataset # unwrap Subset and
↳ random_split
val_indices = val_loader.dataset.indices

```

```

indices_by_class = {i: [] for i in range(10)}
for idx in val_indices:
    _, label = val_dataset[idx]
    indices_by_class[label].append(idx)

def inv_norm(t):
    return (t + 1) / 2

avg_images = class_average_images(val_dataset, indices_by_class, inv_norm)

# Visualize
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    img = avg_images[i].permute(1, 2, 0).cpu().numpy().squeeze()
    ax.imshow(img, cmap='gray')
    ax.set_title(fashion_classes[i], fontsize=9)
    ax.axis('off')
plt.suptitle('Class Average Images (Pixel Space)')
plt.tight_layout()
plt.savefig('figures/fashion_class_averages.png', dpi=100, bbox_inches='tight')
plt.show()

```

1.8 7. Latent Space Analysis: Centroids & t-SNE / PCA

```

[ ]: # Encode validation set
latents_fashion, labels_fashion = encode_dataset(model_fashion, val_loader,
↪device)

print(f"Latents shape: {latents_fashion.shape}")
print(f"Labels shape: {labels_fashion.shape}")

# Compute centroids
centroids_fashion = class_centroids(latents_fashion, labels_fashion)
centroids_array = np.array([centroids_fashion[i] for i in range(10)])
torch.save(torch.from_numpy(centroids_array), 'artifacts/fashion_centroids.pt')
print(f" Saved centroids to artifacts/fashion_centroids.pt")

```

```

[ ]: # PCA visualization
latents_2d_pca, labels_2d_pca = tsne_or_pca(latents_fashion, labels_fashion,
↪method='pca')

plt.figure(figsize=(10, 8))
for i in range(10):
    mask = labels_2d_pca == i
    plt.scatter(latents_2d_pca[mask, 0], latents_2d_pca[mask, 1],
                label=fashion_classes[i], alpha=0.6, s=10)
plt.legend(fontsize=8, markerscale=2)
plt.title('Fashion-MNIST Latent Space (PCA)')

```

```
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('figures/fashion_pca.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
[ ]: # t-SNE visualization (subsample 5k points)
latents_2d_tsne, labels_2d_tsne = tsne_or_pca(latents_fashion, labels_fashion,
                                              method='tsne', max_points=5000)

plt.figure(figsize=(10, 8))
for i in range(10):
    mask = labels_2d_tsne == i
    plt.scatter(latents_2d_tsne[mask, 0], latents_2d_tsne[mask, 1],
                label=fashion_classes[i], alpha=0.6, s=10)
plt.legend(fontsize=8, markerscale=2)
plt.title('Fashion-MNIST Latent Space (t-SNE)')
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('figures/fashion_tsne.png', dpi=150, bbox_inches='tight')
plt.show()
```

1.9 8. Hybrid Image Generation via Centroid Interpolation

Pair: T-shirt/top (class 0) Sneaker (class 7)

```
[ ]: # Centroid-to-centroid interpolation
c0 = centroids_fashion[0] # T-shirt
c7 = centroids_fashion[7] # Sneaker

z_interp = interpolate(c0, c7, steps=11)
hybrids = decode_latents(model_fashion, z_interp, device)

hybrid_strip = grid(hybrids, nrow=11)
save_image(hybrid_strip, 'figures/fashion_hybrid_tshirt_to_sneaker.png')

plt.figure(figsize=(15, 2))
plt.imshow(hybrid_strip, cmap='gray')
plt.title('T-shirt/top → Sneaker (Centroid Interpolation)')
plt.axis('off')
plt.tight_layout()
plt.show()
```

```
[ ]: # Offset hybrids: z0 + (7 - 0)
# Pick a sample near class 0 centroid
class0_mask = labels_fashion == 0
class0_latents = latents_fashion[class0_mask]
dists = np.linalg.norm(class0_latents - c0, axis=1)
nearest_idx = np.argmin(dists)
z0 = class0_latents[nearest_idx]

alphas = np.linspace(0, 1, 11)
offset_imgs = offset_hybrids(model_fashion, z0, c0, c7, alphas, device)

offset_strip = grid(offset_imgs, nrow=11)
save_image(offset_strip, 'figures/fashion_hybrid_offset.png')

plt.figure(figsize=(15, 2))
plt.imshow(offset_strip, cmap='gray')
plt.title('T-shirt/top → Sneaker (Offset Interpolation)')
plt.axis('off')
plt.tight_layout()
plt.show()
```

1.10 9. Challenge: CIFAR-100 — Subset, CAE, Reconstructions, Centroids & Hybrids

Subset (10 classes): bus, rocket, bicycle, motorcycle, train, tank, tractor, lawn_mower, streetcar, pickup_truck

Note: CIFAR-100 does not include airplane/car. We use **bus** **rocket** as example hybrid.

```
[ ]: print(f"CIFAR-100 subset classes: {CIFAR100_SUBSET}")

train_loader_c100, val_loader_c100 = cifar100_loaders(
    batch_size=cifar_args['batch_size'],
    num_workers=cifar_args['num_workers']
)

print(f"Train samples: {len(train_loader_c100.dataset)}")
print(f"Val samples: {len(val_loader_c100.dataset)}")
```

```
[ ]: # Model for CIFAR-100 (RGB, 32x32)
model_cifar = ConvAE(
    in_ch=3,
    latent_dim=cifar_args['latent_dim'],
    img_size=cifar_args['img_size']
).to(device)

print(model_cifar)
print(f"\nTotal params: {sum(p.numel() for p in model_cifar.parameters())},}")
```

```
[ ]: # Train
history_cifar = fit(
    model_cifar,
    train_loader_c100,
    val_loader_c100,
    device,
    epochs=cifar_args['epochs'],
    lr=cifar_args['lr'],
    ckpt_path='checkpoints/best_cifar100_cae.pt'
)

plt.figure(figsize=(8, 4))
plt.plot(history_cifar['train_loss'], label='Train')
plt.plot(history_cifar['val_loss'], label='Val')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.legend()
plt.title('CIFAR-100 Subset Training')
plt.grid(True, alpha=0.3)
plt.savefig('figures/cifar100_loss.png', dpi=100, bbox_inches='tight')
plt.show()

[ ]: # Load best
model_cifar.load_state_dict(torch.load('checkpoints/best_cifar100_cae.pt'))
model_cifar.eval()
print(" Loaded best CIFAR-100 checkpoint")

[ ]: # Reconstructions
val_batch_c100 = next(iter(val_loader_c100))[0][:50].to(device)
with torch.no_grad():
    recon_batch_c100 = model_cifar(val_batch_c100)

combined_c100 = torch.cat([val_batch_c100, recon_batch_c100], dim=0)
recon_grid_c100 = grid(combined_c100, nrow=10)
save_image(recon_grid_c100, 'figures/cifar100_recon_grid.png')

plt.figure(figsize=(10, 8))
plt.imshow(recon_grid_c100)
plt.title('CIFAR-100: Top: Original | Bottom: Reconstructions')
plt.axis('off')
plt.tight_layout()
plt.show()

[ ]: # Latents & centroids
latents_cifar, labels_cifar = encode_dataset(model_cifar, val_loader_c100,
    ↪device)
centroids_cifar = class_centroids(latents_cifar, labels_cifar)
```



```
centroids_array_cifar = np.array([centroids_cifar[i] for i in range(10)])
torch.save(torch.from_numpy(centroids_array_cifar), 'artifacts/
↳cifar100_centroids.pt')
print(f" Saved CIFAR-100 centroids")
```

```
[ ]: # t-SNE
latents_2d_cifar, labels_2d_cifar = tsne_or_pca(latents_cifar, labels_cifar,
                                                method='tsne', max_points=5000)

plt.figure(figsize=(10, 8))
for i in range(10):
    mask = labels_2d_cifar == i
    plt.scatter(latents_2d_cifar[mask, 0], latents_2d_cifar[mask, 1],
                label=CIFAR100_SUBSET[i], alpha=0.6, s=10)
plt.legend(fontsize=8, markerscale=2)
plt.title('CIFAR-100 Subset Latent Space (t-SNE)')
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('figures/cifar100_tsne_subset.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
[ ]: # Hybrid: bus (idx 0) rocket (idx 1)
c_bus = centroids_cifar[0]
c_rocket = centroids_cifar[1]

z_interp_cifar = interpolate(c_bus, c_rocket, steps=11)
hybrids_cifar = decode_latents(model_cifar, z_interp_cifar, device)

hybrid_strip_cifar = grid(hybrids_cifar, nrow=11)
save_image(hybrid_strip_cifar, 'figures/cifar100_hybrid_bus_to_rocket.png')

plt.figure(figsize=(15, 2))
plt.imshow(hybrid_strip_cifar)
plt.title('Bus → Rocket (CIFAR-100 Centroid Interpolation)')
plt.axis('off')
plt.tight_layout()
plt.show()
```

1.11 10. Report: Difficulties, Mitigations, Method, Limitations, Extensions

1.11.1 Difficulties (Methodological)

Blurry reconstructions: MSE loss encourages pixel-wise averaging, leading to blurred outputs especially for complex textures in CIFAR-100. High-frequency details are smoothed out.

Centroid instability: Class centroids computed as simple means can be unstable when class distributions are multimodal or imbalanced. Some classes in CIFAR-100 subset have fewer samples,

leading to noisier centroids.

Entangled latents: Without explicit disentanglement objectives, latent dimensions capture correlated attributes. Interpolation can yield non-semantic transitions (e.g., simultaneous changes in shape, texture, color).

CIFAR-100 complexity: RGB 32×32 images with diverse textures/colors require larger latent capacity and deeper architectures. Training converges slower than Fashion-MNIST.

1.11.2 Mitigations

Larger latent dimension: Increased from 64 (Fashion-MNIST) to 128 (CIFAR-100) to capture richer representations.

Additional conv block: Added $64\rightarrow 128$ channel block for CIFAR-100 encoder to handle RGB complexity.

Longer training: Extended CIFAR-100 epochs to 20 to ensure convergence despite smaller subset size.

Class-balanced sampling (not implemented but recommended): Could stratify batches to ensure equal class representation during training.

Dimensionality reduction (PCA/t-SNE): Subsampled to 5k points for t-SNE to reduce computation while preserving cluster structure.

1.11.3 Method Overview

We built symmetric convolutional autoencoders with Tanh output activation and $[-1,1]$ normalization. For Fashion-MNIST (28×28 grayscale), encoder downsamples via two 4×4 stride-2 convolutions to 7×7 feature maps, then flattens to latent vector. Decoder mirrors this with transposed convolutions. For CIFAR-100 (32×32 RGB), we add a third conv block to reach 4×4 spatial resolution before latent bottleneck.

Class centroids are computed as arithmetic means of latent vectors per class on validation set. Hybrid images are generated by linearly interpolating between centroids in latent space, then decoding each intermediate point. We also tested offset hybrids: selecting a real sample near one centroid and interpolating the direction vector to another centroid.

Latent space structure is visualized via PCA (fast, linear) and t-SNE (nonlinear, captures local structure). PCA reveals global variance directions; t-SNE shows cluster separability.

1.11.4 Limitations

Qualitative evaluation only: No quantitative metrics (FID, IS) for reconstruction quality or hybrid realism.

Centroid = mean: Assumes Gaussian class distributions; fails for multimodal classes. K-means per class would find multiple prototypes.

Linear interpolation: Straight-line paths in latent space may traverse low-density regions, yielding unrealistic intermediates. Geodesic or learned paths would be better.

Hybrid depends on separability: Classes with overlapping latent clusters (e.g., similar clothing items) produce less interpretable hybrids.

CIFAR-100 class heterogeneity: Vehicle classes vary widely in pose, color, background. Single centroid may not represent class well.

No perceptual loss: MSE penalizes pixel differences equally; perceptual loss (VGG features) would preserve semantic content better.

1.11.5 Extensions

-VAE for disentanglement: Enforce independence of latent factors via weighted KL term. Enables controlled attribute editing (e.g., color vs shape).

Perceptual loss: Replace/augment MSE with VGG feature matching or LPIPS to improve texture fidelity.

Class-conditional decoder: Separate decoder per class or use class embedding to condition generation. Better handles intra-class variation.

Multiple prototypes per class: K-means clustering within each class to find k centroids. Interpolate between nearest prototypes.

Style/content disentanglement: AdaIN-based architectures to separate style (texture, color) from content (shape). Enables style transfer between classes.

Quantitative evaluation: Compute FID on reconstructions, measure centroid stability across random seeds, evaluate hybrid realism via human study or classifier confidence.

Larger CIFAR-100 subset or full set: Current 10-class subset is tractable but limits generalization. Full 100-class would test scalability.

Attention mechanisms: Spatial attention in decoder to focus reconstruction on salient regions.

Progressive training: Start with low-resolution, gradually increase to full size (ProGAN-style). Stabilizes training for complex datasets.

1.12 11. Reproducibility & Artifacts Summary

```
[ ]: import os

print("\n" + "="*60)
print("ARTIFACTS SUMMARY")
print("="*60)

print("\nCheckpoints:")
for f in sorted(os.listdir('checkpoints')):
    path = os.path.join('checkpoints', f)
    size = os.path.getsize(path) / (1024**2)
    print(f"  {os.path.abspath(path)} ({size:.2f} MB)")

print("\nFigures:")
for f in sorted(os.listdir('figures')):
```

```

        if f.endswith('.png'):
            print(f"  {os.path.abspath(os.path.join('figures', f))}")

print("\nArtifacts:")
for f in sorted(os.listdir('artifacts')):
    if f.endswith('.pt'):
        print(f"  {os.path.abspath(os.path.join('artifacts', f))}")

print("\n" + "="*60)
print(f"Environment: PyTorch {torch.__version__}, Device: {device}")
print("="*60)

```